Lab 4: VibeCoding - Chat History

Repo: SETAPESU26/58_color_pattern (Memory Color Pattern Repair Lab)
Tool: Claude (Sonnet 5.5) | Session date: 22 Oct 2026
This is a log of the whole session up to the point of export. Student messages are verbatim; assistant replies and tool work are condensed. Code changes are shown as the exact git diffs of each commit.

Student

[Attached: Lab 4 VibeCoding student handout PDF]
https://github.com/SETAPESU26/58_color_pattern

Upload the following to your Lab repo under Lab-4 folder,
a. Videos (before and after)
b. Updated Code
c. Chat history exported as a doc/pdf

Refer worksheet for the code assigned to you

Claude - Step 1: read the repo

Fetched the repo README and the three source files (main.py, game/color_button.py, game/game_engine.py).
README summary: a Simon-style Pygame memory game with one deliberate bug and three optional features:
Task 1: Fix the sequence duplication bug
Task 2: Dynamic playback acceleration
Task 3: Sound effects / audio frequencies per pad
Task 4: Player turn input timer bar
Root cause of the Task 1 bug found in start_next_round(): `self.sequence += self.sequence + [new_color]` doubles the pattern every round.

Claude - Task 1 - Fix sequence duplication bug

Replaced the doubling line with a single append so each round adds exactly one color.

Commit 6d9deeb: Task 1: Fix sequence duplication bug (append one step per round)

```
diff --git a/game/game_engine.py b/game/game_engine.py
index 7dd2fc4..a4526a6 100644
--- a/game/game_engine.py
+++ b/game/game_engine.py
@@ -43,7 +43,7 @@ class GameEngine:
     def start_next_round(self):
         new_color = random.randint(0, 3)

-        self.sequence += self.sequence + [new_color]
+        self.sequence.append(new_color)

         self.player_input.clear()
         self.state = "WATCH"
```

Claude - Task 2 - Dynamic playback acceleration

Flash time starts at 450 ms and shrinks by 30 ms per round (min 150 ms). Pause starts at 200 ms and shrinks by 14 ms per round (min 60 ms). Values are class constants so they are easy to tune.

Commit d77cccf: Task 2: Dynamic playback acceleration across rounds

```
diff --git a/game/game_engine.py b/game/game_engine.py
index a4526a6..5d57fea 100644
--- a/game/game_engine.py
+++ b/game/game_engine.py
@@ -4,6 +4,14 @@ from game.color_button import ColorButton
```

```
 class GameEngine:
+    # Playback speed settings (milliseconds)
+    BASE_FLASH = 450
+    BASE_PAUSE = 200
+    MIN_FLASH = 150
+    MIN_PAUSE = 60
+    FLASH_STEP = 30   # flash gets this much shorter each round
+    PAUSE_STEP = 14   # pause gets this much shorter each round
+
     def __init__(self, width, height):
         self.width = width
         self.height = height
@@ -27,8 +35,8 @@ class GameEngine:
         self.state = "WATCH"
         self.showing_step = 0
         self.step_start_time = 0
-        self.flash_duration = 450
-        self.pause_duration = 200
+        self.flash_duration = self.BASE_FLASH
+        self.pause_duration = self.BASE_PAUSE
         self.is_flashing = False

         self.player_lit_button = None
@@ -45,6 +53,11 @@ class GameEngine:

         self.sequence.append(new_color)

+        # Speed up playback as rounds advance (never faster than the minimums)
+        round_num = len(self.sequence)
+        self.flash_duration = max(self.MIN_FLASH, self.BASE_FLASH - (round_num - 1) * self.FLASH_STEP)
+        self.pause_duration = max(self.MIN_PAUSE, self.BASE_PAUSE - (round_num - 1) * self.PAUSE_STEP)
+
         self.player_input.clear()
         self.state = "WATCH"
         self.showing_step = 0
```

Claude - Task 3 - Sound per pad

Generated sine-wave tones in code (no audio files): Red C4 261.63 Hz, Blue E4 329.63 Hz, Green G4 392.00 Hz, Yellow C5 523.25 Hz. Played when a pad flashes during playback and when the player clicks it. Falls back to silent if audio is unavailable.

Commit 2440b67: Task 3: Add a distinct audio pitch per colored pad

```
diff --git a/game/game_engine.py b/game/game_engine.py
index 5d57fea..ed1a86b 100644
--- a/game/game_engine.py
+++ b/game/game_engine.py
@@ -1,7 +1,25 @@
+import array
+import math
 import random
 import pygame
 from game.color_button import ColorButton

+# One pitch per pad: Red=C4, Blue=E4, Green=G4, Yellow=C5 (a C major arpeggio)
+PAD_FREQUENCIES = [261.63, 329.63, 392.00, 523.25]
+SAMPLE_RATE = 44100
+
+
+def make_tone(freq, duration_ms=250, volume=0.4):
+    """Build a sine-wave pygame Sound with a short fade in/out to avoid clicks."""
+    total = int(SAMPLE_RATE * duration_ms / 1000)
+    fade = int(SAMPLE_RATE * 0.01)
+    samples = array.array("h")
+    for i in range(total):
+        envelope = min(1.0, i / fade, (total - i) / fade)
+        value = int(32767 * volume * envelope * math.sin(2 * math.pi * freq * i / SAMPLE_RATE))
+        samples.append(value)
+    return pygame.mixer.Sound(buffer=samples.tobytes())
+

 class GameEngine:
     # Playback speed settings (milliseconds)
@@ -46,8 +64,29 @@ class GameEngine:
         self.font_title = pygame.font.SysFont(None, 40)
         self.font_medium = pygame.font.SysFont(None, 28)
+        self.sounds = self.load_sounds()
```

```
+
         self.start_next_round()

+    def load_sounds(self):
+        """Create one tone per pad. Returns an empty list if audio is unavailable."""
+        try:
+            # Mono, 16-bit so the generated samples match the mixer format
+            pygame.mixer.quit()
+            pygame.mixer.init(SAMPLE_RATE, -16, 1, 512)
+            return [make_tone(f) for f in PAD_FREQUENCIES]
+        except pygame.error:
+            return []
+
+    def play_sound(self, color_id):
+        if self.sounds:
+            self.sounds[color_id].play()
+
+    def light_button(self, color_id):
+        """Light a pad and play its tone (used for both playback and clicks)."""
+        self.buttons[color_id].is_lit = True
+        self.play_sound(color_id)
+
     def start_next_round(self):
         new_color = random.randint(0, 3)

@@ -63,7 +102,7 @@ class GameEngine:
         self.showing_step = 0
         self.step_start_time = pygame.time.get_ticks()
         self.is_flashing = True
-        self.buttons[self.sequence[0]].is_lit = True
+        self.light_button(self.sequence[0])

     def update(self):
         now = pygame.time.get_ticks()
@@ -86,7 +125,7 @@ class GameEngine:
                     self.showing_step += 1
                     if self.showing_step < len(self.sequence):
                         next_id = self.sequence[self.showing_step]
-                        self.buttons[next_id].is_lit = True
+                        self.light_button(next_id)
                         self.is_flashing = True
                         self.step_start_time = now
                     else:
@@ -101,7 +140,7 @@ class GameEngine:
         if self.state == "PLAYER_TURN" and event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
             for btn in self.buttons:
                 if btn.contains(event.pos):
-                    btn.is_lit = True
+                    self.light_button(btn.color_id)
                     self.player_lit_button = btn
                     self.player_lit_start = pygame.time.get_ticks()
```

Claude - Task 4 - Player turn timer bar

Countdown bar under the grid (green to yellow to red). Limit starts at 5 s and drops 100 ms per round (min 2.5 s), and restarts after each correct click. Expiry triggers Game Over with a 'TIME'S UP!' message.

Commit ab2c62c: Task 4: Add player turn countdown timer bar

```
diff --git a/game/game_engine.py b/game/game_engine.py
index ed1a86b..aa2fef1 100644
--- a/game/game_engine.py
+++ b/game/game_engine.py
@@ -30,6 +30,11 @@ class GameEngine:
     FLASH_STEP = 30   # flash gets this much shorter each round
     PAUSE_STEP = 14   # pause gets this much shorter each round

+    # Player turn timer (milliseconds). The clock restarts after every correct click.
+    BASE_TURN_TIME = 5000
+    MIN_TURN_TIME = 2500
+    TURN_TIME_STEP = 100  # less time per click as rounds advance
+
     def __init__(self, width, height):
         self.width = width
         self.height = height
@@ -61,6 +66,10 @@ class GameEngine:
         self.player_lit_start = 0
         self.player_flash_duration = 150
```

```
+        self.turn_start_time = 0
+        self.turn_time_limit = self.BASE_TURN_TIME
+        self.game_over_reason = ""
+
         self.font_title = pygame.font.SysFont(None, 40)
         self.font_medium = pygame.font.SysFont(None, 28)

@@ -130,6 +139,16 @@ class GameEngine:
                         self.step_start_time = now
                     else:
                         self.state = "PLAYER_TURN"
+                        self.turn_time_limit = max(
+                            self.MIN_TURN_TIME,
+                            self.BASE_TURN_TIME - (len(self.sequence) - 1) * self.TURN_TIME_STEP,
+                        )
+                        self.turn_start_time = now
+
+        elif self.state == "PLAYER_TURN":
+            if now - self.turn_start_time >= self.turn_time_limit:
+                self.game_over_reason = "TIME'S UP! GAME OVER"
+                self.state = "GAME_OVER"

     def handle_event(self, event):
         if self.state == "GAME_OVER":
@@ -152,12 +171,16 @@ class GameEngine:
         current_idx = len(self.player_input) - 1

         if self.player_input[current_idx] != self.sequence[current_idx]:
+            self.game_over_reason = "WRONG PATTERN! GAME OVER"
             self.state = "GAME_OVER"
             return

         if len(self.player_input) == len(self.sequence):
             self.score += 1
             self.start_next_round()
+        else:
+            # Correct click, pattern not finished yet: restart the countdown
+            self.turn_start_time = pygame.time.get_ticks()

     def reset(self):
         self.sequence.clear()
@@ -168,6 +191,28 @@ class GameEngine:
         self.player_lit_button = None
         self.start_next_round()

+    def render_timer_bar(self, screen):
+        bar_width = 300
+        bar_height = 16
+        bar_x = self.width // 2 - bar_width // 2
+        bar_y = 460
+
+        elapsed = pygame.time.get_ticks() - self.turn_start_time
+        remaining = max(0.0, 1.0 - elapsed / self.turn_time_limit)
+
+        # Green -> yellow -> red as time runs out
+        if remaining > 0.5:
+            color = (80, 240, 130)
+        elif remaining > 0.25:
+            color = (255, 220, 80)
+        else:
+            color = (240, 70, 70)
+
+        pygame.draw.rect(screen, (50, 54, 64), (bar_x, bar_y, bar_width, bar_height), border_radius=8)
+        if remaining > 0:
+            pygame.draw.rect(screen, color, (bar_x, bar_y, int(bar_width * remaining), bar_height), border_radius=8)
+        pygame.draw.rect(screen, (70, 75, 85), (bar_x, bar_y, bar_width, bar_height), width=2, border_radius=8)
+
     def render(self, screen):
         screen.fill((22, 24, 30))

@@ -185,12 +230,15 @@ class GameEngine:
         for btn in self.buttons:
             btn.render(screen)

+        if self.state == "PLAYER_TURN":
+            self.render_timer_bar(screen)
+
         if self.state == "GAME_OVER":
             overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
             overlay.fill((0, 0, 0, 200))
```

```
            screen.blit(overlay, (0, 0))

-           over_surf = self.font_title.render("WRONG PATTERN! GAME OVER", True, (240, 70, 70))
+           over_surf = self.font_title.render(self.game_over_reason or "GAME OVER", True, (240, 70, 70))
            screen.blit(over_surf, (self.width // 2 - over_surf.get_width() // 2, self.height // 2 - 40))

            final_score_surf = self.font_medium.render(f"Final Score: {self.score}", True, (255, 255, 255))
```

Claude - Verification

Ran the game logic headlessly with simulated clicks and a fake clock. Output:

```
sounds loaded: 4
round 1: len=1 flash=450 pause=200 turn_limit=5000
round 2: len=2 flash=420 pause=186 turn_limit=4900
round 3: len=3 flash=390 pause=172 turn_limit=4800
round 4: len=4 flash=360 pause=158 turn_limit=4700
round 5: len=5 flash=330 pause=144 turn_limit=4600
round 6: len=6 flash=300 pause=130 turn_limit=4500
round 7: len=7 flash=270 pause=116 turn_limit=4400
round 8: len=8 flash=240 pause=102 turn_limit=4300
score: 8
after timeout: GAME_OVER | TIME'S UP! GAME OVER
after R: len 1 score 0 state WATCH
after wrong click: GAME_OVER | WRONG PATTERN! GAME OVER
timer reset on correct click, still alive: True
```

Claude - Hand-off

Delivered the final game_engine.py and lab4_commits.patch (the four commits, applied with `git am`). Explained how to record the before/after videos and push to the Lab repo. Noted the chat export is a graded deliverable and suggested exporting the full conversation from the chat menu.

Student

i will do the screenrecording part
you do the rest

Claude - Packaging

Built the Lab-4 folder: code/ (main.py, game/, lab4_commits.patch), videos/ placeholder, and a chat-history PDF, and sent it as Lab-4.zip. Said that videos were the student's part and pushing needed their repo.

Student

https://github.com/SunayHegde/SE-LAB this the repo to push into. Tell me when to take the before video

Claude - Push attempt and before-video timing

Tried to attach SunayHegde/SE-LAB with push access; it failed with 'link your GitHub account to let Claude access repositories', so nothing was pushed. Told the student to link GitHub under Settings > Connectors or push manually. Said the before video must be recorded on the original code: clone SETAPESU26/58_color_pattern, pip install pygame, python main.py, record, and don't copy the fixed files in until done. Gave manual git push steps for SE-LAB.

Student

tell me what should i take the video of

Claude - Video shot list

Before video (original code): watch the first pattern, click it correctly, then let round 2 replay so the pattern length jumps (the bug); original is silent, one speed, no timer. After video (fixed code): play rounds 1-3 with one-step growth and distinct pad pitches, show the flashes speeding up, show the timer bar draining green to red, and let it expire to show the 'TIME'S UP!' screen. Tips: capture system audio, click carefully.

Student

ok so i oponed lab4 zip in vscode now next what too do

Claude - Next steps

Gave an ordered list: clone the original repo to a folder called 'original' and record the before video there; run the fixed code from Lab-4/code for the after video; move the videos into Lab-4/videos; clone SunayHegde/SE-LAB, copy Lab-4 in, git add / commit / push.

Student

can u fix the code so i can record after

Claude - Code check

Checked that Lab-4/code matched the tested version (identical, compiles, all four changes present). Told the student the code was already fixed and listed likely causes if something went wrong (running the original folder by mistake, pygame not installed, python vs python3, audio/system volume) and asked for the exact error.

Student

ok ive done taking the 2 videos. now can u please update my github

Claude - GitHub push

Tried again to attach SE-LAB; still 'link your GitHub account'. Explained it could not push, and that the videos are on the student's machine anyway. Gave push steps (clone SE-LAB, copy Lab-4, git add/commit/push, verify in the browser) and common fixes (login/token, 100 MB file limit).

Student

[Attached: Lab-4.zip] check if this is fine?

Claude - Review of the submission zip

Unzipped and checked. Code identical to the tested version; PDF, Before.mp4 and After.mp4 present with audio. Before.mp4 was 17.8 s and After.mp4 30.7 s (handout asks for 10 s). From contact-sheet frames: After started on a stale 'TIME'S UP' screen from a previous run, showed the timer bar and score reaching 6; Before showed score 0 to 1 then Game Over but did not clearly show the pattern-length bug. After had audible tones; Before was silent as expected. Recommended trimming both to about 10 s and deleting videos/README.txt.

Student

[Attached: After.mp4] Can u trim the video from 7 to 17 seconds

Claude - Trim After

Trimmed with ffmpeg (-ss 7 -t 10) to a 10.03 s clip with audio, verified frames (fresh game, timer bar draining, score 0 to 3), and sent After_trimmed.mp4. Noted the 'TIME'S UP!' screen is no longer included.

Student

[Attached: Before.mp4] 5 to 15 for before

Claude - Trim Before

Trimmed with ffmpeg (-ss 5 -t 10) to a 10.03 s clip, verified frames (score 0 to 1, round 2 'Watch the pattern...', then Game Over at final score 1), and sent Before_trimmed.mp4. Noted it does not clearly show the pattern-length bug.

Student

[Attached: Lab_4.zip] I have changed the video. dont mind before not being correct. other than that everything is correct right? mainly the code and pdf

Claude - Final check

Unzipped the new zip (folder named 'Lab 4'). code/ identical to the tested version, PDF identical to the earlier one, both videos exactly 10.03 s with audio, README.txt removed. Suggested renaming the folder to 'Lab-4' as in the handout and exporting the full conversation, since the earlier PDF only covered the session up to its creation.

Student

give the conversation up till now in pdf

Claude - This document

Regenerated the chat history as this PDF, covering the whole session up to this request.